Project Name: CleanOps

Team Members:

1. Vio Gustian Nur Alamsyah
2. Nur Aji Ananta

Problem (The "Why")

Maintaining floor cleanliness in places such as supermarkets, offices, and public facilities can be challenging because unexpected incidents may occur at any time. For example, food or drinks may spill, or objects and waste may fall onto the floor.

Currently, these incidents are usually discovered when a cleaning staff member notices them directly or when someone manually reports the issue. This can cause delays in responding to the problem and may leave the affected area dirty for an extended period of time.

- **Current Issue:** There is no automated system that can quickly detect and report dirty floor conditions based on CCTV monitoring.
- **Impact:** Visitors and employees may be disturbed by dirty or unsafe areas, while cleaning staff may need additional time to discover the problem and determine which area should be handled first.

Proposed Solution

CleanOps is a computer vision-based floor cleanliness monitoring and response system that uses CCTV cameras to detect dirty conditions in monitored floor areas.

The CCTV camera will be connected to the system, and its monitored area will be mapped onto a digital floor map. When the computer vision model detects an object or condition categorized as dirt, the system identifies its location on the floor map and generates an alert for the user.

The system also provides multiple cleaning units that can be registered and assigned to specific areas. When a cleaning alert is generated, the system calculates the distance between the detected location and available cleaning units and recommends the nearest unit to handle the incident.

To keep the project achievable within two weeks, physical autonomous cleaning robots will not be included in the MVP. Instead, cleaning units will be represented as virtual units within the system. Physical robot integration can be considered as a future improvement.

- **Our Approach:**

1. The user registers a CCTV camera and maps its monitored area onto a digital floor map.
2. The system receives video or image data from the CCTV camera.
3. A computer vision model detects objects or conditions categorized as dirt.
4. The detected position is converted into coordinates on the floor map.
5. The system generates a cleaning alert containing the detected location and timestamp.
6. The system calculates and identifies the nearest available cleaning unit.
7. The user can monitor the incident and assigned cleaning unit through the web application.

- **Key Value:** The system changes the cleaning process from a **manual and reactive process** into a more **automated and responsive process**. Cleaning staff can receive information about potential cleaning incidents without continuously monitoring the entire area themselves.

Core Features

- **Feature 1: CCTV Floor Mapping & Monitoring**
  Users can register CCTV cameras and map their monitored areas onto a digital floor map. The system displays the monitored area and its coordinates, which are used to determine the location of detected incidents.
- **Feature 2: AI-Based Dirt Detection & Alert**
  The system uses a computer vision model to detect objects or conditions

categorized as dirt on the monitored floor area. When a detection occurs, the system generates a cleaning alert containing the location and time of the incident.

- **Feature 3: Cleaning Unit Assignment**
  Users can register multiple cleaning units and their current positions. When a cleaning alert is generated, the system calculates the distance between available cleaning units and the detected location and recommends the nearest unit to handle the incident.

High-Level Architecture & Quality Assurance

- **Architecture Pattern:** Layered Architecture
- **Frontend:** React
- **Backend:** ASP.NET Core Web API using .NET SDK 8
- **Database:** PostgreSQL
- **Computer Vision:** YOLO-based object detection using a pre-trained or fine-tuned model, with inference potentially implemented using ONNX Runtime or a dedicated inference service.
- **Communication:** REST API between React and ASP.NET Core. SignalR may be used for real-time cleaning alerts and status updates.
- **Design Patterns Used:**
  - Dependency Injection
  - Repository Pattern
  - Service Layer
  - Strategy Pattern for cleaning unit assignment
  - DTO Pattern for API communication
- **Code Quality Goal:** We commit to maintaining a green Jenkins pipeline, writing unit tests to be tracked via Cobertura, and keeping Qodana static analysis warnings below 10 (Target: 0).

Identified Challenges & Risks

- **Risk 1: Computer Vision Accuracy**
  The YOLO model may produce false positives or false negatives depending on lighting conditions, camera angles, object appearance, and the quality of the training data.

- **Risk 2: Camera-to-Floor Coordinate Mapping**
  Converting an object's position from the CCTV camera perspective into coordinates on the digital floor map may require proper camera calibration and mapping.
- **Risk 3: Real-Time Processing Performance**
  Processing video streams in real time can require significant computing resources. For the MVP, the number of cameras and processing frame rate may need to be limited to maintain stable performance.
- **Risk 4: Cleaning Unit and Hardware Availability**
  A physical autonomous cleaning unit may not be available during the project development and prototyping phase. To address this limitation, the cleaning unit can be simulated digitally within the web application using **Three.js**. The simulation can represent the cleaning unit's position, movement, destination, and assigned cleaning area on a virtual 2D/3D floor map.

Additional note:
To ensure that the project can be completed within two weeks, the MVP will focus on the following software workflow:

CCTV → AI Detection → Floor Coordinate → Cleaning Alert → Nearest Cleaning Unit

The cleaning unit assignment will initially be implemented as a software-based simulation. If a physical cleaning unit or robot is not available for the prototype, Three.js will be used to simulate the unit's movement within the mapped environment.

The Three.js simulation can demonstrate how the system would work in a real-world scenario, including:

1. Displaying the floor layout and detected dirty area.
2. Displaying available cleaning units on the virtual map.
3. Selecting the nearest available cleaning unit.
4. Assigning the cleaning unit to the detected location.
5. Simulating the unit's movement toward the target location.

6. Updating the unit status after reaching the target location.

This approach allows the team to demonstrate the complete system workflow without requiring physical robot hardware during the two-week development period.

Autonomous robot hardware, advanced path planning, and physical robot integration are outside the scope of the MVP.

In the future, the Three.js simulation can be replaced or extended with real hardware integration:

CCTV → AI Detection → Floor Coordinate → Alert → Nearest Robot → Path Planning → Autonomous Cleaning